Top 60 Apache Spark Intermediate & Advanced MCQs with Solutions

---

Section A – Spark Core & RDD (1–15)

Q1. Spark stores intermediate data in memory using:
A. HDFS
B. RDD lineage
C. Checkpointing
D. Shuffle files

✅ **Answer:** B. RDD lineage
👉 Lineage allows recomputation of lost partitions instead of storing every intermediate dataset.

Q2. Which Spark operation is *lazy*?
A. collect()
B. count()
C. filter()
D. foreach()

✅ **Answer:** C. filter()
👉 Transformations like `map`, `filter` are lazy, actions like `collect`, `count` trigger execution.

Q3. Which of the following creates a **narrow dependency**?
A. groupByKey
B. reduceByKey
C. join
D. map

✅ **Answer:** D. map
👉 Narrow dependencies mean each parent partition is used by at most one child (e.g., `map`).

Q4. In Spark, **checkpointing** is mainly used for:
A. Performance improvement

B. Fault tolerance
C. Reducing shuffle size
D. Improving caching

✅ **Answer:** B. Fault tolerance
👉 Checkpointing saves RDD to reliable storage (e.g., HDFS).

Q5. Which is TRUE about Spark RDD persistence levels?
A. MEMORY_ONLY stores objects as deserialized JVM objects
B. MEMORY_AND_DISK stores only serialized format
C. DISK_ONLY keeps RDD cached in memory
D. NONE of the above

✅ **Answer:** A. MEMORY_ONLY stores objects as deserialized JVM objects

---

Q6. In Spark, which operation triggers a **shuffle**?
A. map
B. filter
C. flatMap
D. groupByKey

✅ **Answer:** D. groupByKey
👉 Wide operations (groupByKey, join, reduceByKey) require shuffle.

Q7. Which of these is a valid Spark action?
A. mapValues()
B. saveAsTextFile()
C. flatMap()
D. filter()

✅ **Answer:** B. saveAsTextFile()

Q8. RDDs are:
A. Immutable and distributed
B. Mutable and distributed
C. Mutable and local
D. None

✅ **Answer:** A. Immutable and distributed

Q9. Which is the most efficient for aggregations?
A. groupByKey
B. reduceByKey
C. map
D. collect

✅ **Answer:** B. reduceByKey
👉 `groupByKey` can cause huge shuffle data, `reduceByKey` aggregates before shuffle.

Q10. RDD checkpoint removes:
A. Cached data
B. Lineage
C. Shuffled data
D. Partitions

✅ **Answer:** B. Lineage

Q11. Spark's DAG Scheduler splits jobs into:
A. Blocks
B. Stages
C. Threads
D. Executors

✅ **Answer:** B. Stages

Q12. The **driver program** in Spark is responsible for:
A. Executing transformations
B. Scheduling tasks
C. Storing partitions
D. Only data shuffling

✅ **Answer:** B. Scheduling tasks

Q13. The default partitioner in Spark for pair RDDs is:
A. RangePartitioner
B. HashPartitioner
C. CustomPartitioner
D. None

✅ **Answer:** B. HashPartitioner

Q14. Persisting an RDD with `.persist(StorageLevel.DISK_ONLY)` will:
A. Store in memory
B. Store in HDFS
C. Store in local disk of executor nodes
D. None

✅ **Answer:** C. Store in local disk of executor nodes

Q15. Which Spark API is **type-safe**?
A. RDD
B. DataFrame
C. Dataset
D. SQL

✅ **Answer:** C. Dataset

Section B – Spark SQL & DataFrames (16–30)

Q16. The **Catalyst optimizer** is used in:
A. Spark Streaming
B. Spark SQL
C. Spark Core
D. MLlib

✅ **Answer:** B. Spark SQL

Q17. Tungsten project in Spark focuses on:
A. Memory management & code generation
B. Faster shuffles

C. Streaming
D. Machine learning

✅ **Answer:** A. Memory management & code generation

Q18. Spark SQL uses which execution model?
A. Interpreter
B. Vectorized execution
C. Row-by-row execution
D. None

✅ **Answer:** B. Vectorized execution

Q19. A **DataFrame** in Spark is:
A. Distributed collection of Java objects
B. Distributed collection of rows with schema
C. Local Pandas DataFrame
D. None

✅ **Answer:** B. Distributed collection of rows with schema

Q20. Spark SQL joins are optimized using:
A. Broadcast joins
B. Sort merge joins
C. Shuffle hash joins
D. All of the above

✅ **Answer:** D. All of the above

Q21. Which is faster for small lookup tables?
A. Shuffle join
B. Broadcast join
C. Cartesian join
D. Sort merge join

✅ **Answer:** B. Broadcast join

Q22. Spark SQL query plan has stages:
A. Parsed → Analyzed → Optimized → Physical
B. Optimized → Parsed → Executed
C. Analyzed → Executed
D. None

✅ **Answer:** A. Parsed → Analyzed → Optimized → Physical

Q23. Which Spark SQL function returns top-N rows?
A. limit()
B. head()
C. show()
D. take()

✅ **Answer:** A. limit()

Q24. Catalyst optimizer uses:
A. Rule-based optimization
B. Cost-based optimization
C. Both
D. None

✅ **Answer:** C. Both

Q25. Spark DataFrames are:
A. Schema-less
B. Schema-based
C. Partitions without metadata
D. None

✅ **Answer:** B. Schema-based

Q26. Tungsten avoids Java object overhead by:
A. Off-heap memory
B. Lazy evaluation
C. Hash partitioning
D. Serialization

✅ **Answer:** A. Off-heap memory

Q27. Spark SQL supports:
A. Hive UDFs
B. Custom UDFs
C. Built-in functions
D. All of the above

✅ **Answer:** D. All of the above

Q28. Spark DataSet API is available in:
A. Java, Scala
B. Python
C. R
D. None

✅ **Answer:** A. Java, Scala

Q29. Which command registers a DataFrame as a SQL temporary view?
A. createOrReplaceTempView
B. registerSQL
C. cacheTable
D. tableRegister

✅ **Answer:** A. createOrReplaceTempView

Q30. Spark SQL integrates with Hive using:
A. HiveContext / SparkSession with Hive support
B. HiveLoader
C. HiveExecutor
D. None

✅ **Answer:** A. HiveContext / SparkSession with Hive support

---

Section C – Spark Streaming & Structured Streaming (31–45)

Q31. Spark Streaming works on:
A. True streaming
B. Micro-batch processing
C. Event-by-event processing
D. Continuous processing only

✅ **Answer:** B. Micro-batch processing

Q32. Structured Streaming supports which output modes?
A. Append
B. Update
C. Complete
D. All of the above

✅ **Answer:** D. All of the above

Q33. Which source is supported in Structured Streaming?
A. Kafka
B. File system
C. Socket
D. All

✅ **Answer:** D. All

Q34. Watermarking in Spark Streaming is used for:
A. Windowing
B. Handling late data
C. Join optimization
D. None

✅ **Answer:** B. Handling late data

Q35. The default checkpoint directory for streaming jobs must be:
A. Local disk

B. HDFS / reliable storage
C. RAM
D. None

✅ **Answer:** B. HDFS / reliable storage

Q36. In Spark Structured Streaming, trigger interval defines:
A. Partition size
B. Shuffle frequency
C. Micro-batch execution frequency
D. None

✅ **Answer:** C. Micro-batch execution frequency

Q37. Kafka integration with Spark uses:
A. Direct stream approach
B. Receiver-based approach
C. Both (depending on version)
D. None

✅ **Answer:** C. Both

Q38. Structured Streaming engine is:
A. Continuous event processor
B. Incremental query engine
C. Batch-only
D. None

✅ **Answer:** B. Incremental query engine

Q39. Windowed aggregations require:
A. Event-time column
B. Shuffle partitions
C. Cache
D. None

✅ **Answer:** A. Event-time column

Q40. Which format is NOT directly supported in streaming?
A. JSON
B. Parquet
C. Avro
D. Excel

✅ **Answer:** D. Excel

Q41. Streaming state store keeps:
A. Off-heap memory only
B. Aggregated intermediate state
C. Cache of all input
D. None

✅ **Answer:** B. Aggregated intermediate state

Q42. Checkpointing in streaming stores:
A. Offsets & operator state
B. Only metadata
C. Only RDD lineage
D. None

✅ **Answer:** A. Offsets & operator state

Q43. Continuous processing mode (since Spark 2.3) reduces:
A. Latency
B. Throughput
C. Shuffle
D. Executor count

✅ **Answer:** A. Latency

Q44. Kafka offset commits in Spark are stored in:
A. ZooKeeper
B. Kafka itself
C. Checkpoint directory
D. Both B & C

✅ **Answer:** D. Both B & C

Q45. Structured Streaming is built on top of:
A. RDDs
B. Dataset/DataFrame API
C. Hadoop MapReduce
D. None

✅ **Answer:** B. Dataset/DataFrame API

Section D – Spark Performance & Cluster Management (46–60)

Q46. Spark job parallelism is primarily controlled by:
A. Number of executors
B. Number of cores per executor
C. Number of partitions
D. All

✅ **Answer:** D. All

Q47. Increasing `spark.sql.shuffle.partitions` affects:
A. Number of tasks post shuffle
B. Number of jobs
C. Cache size
D. None

✅ **Answer:** A. Number of tasks post shuffle

Q48. Broadcast variables are used to:
A. Cache RDDs
B. Send small lookup tables to all executors
C. Broadcast DataFrames
D. None

✅ **Answer:** B. Send small lookup tables to all executors

Q49. Accumulators are:
A. Write-only variables for workers
B. Read-only variables for workers
C. Mutable shared variables
D. None

✅ **Answer:** A. Write-only variables for workers

Q50. Skewed joins can be optimized using:
A. Salting
B. Broadcast join
C. Skew hints
D. All

✅ **Answer:** D. All

Q51. Spark fair scheduling allows:
A. Multiple jobs to share resources fairly
B. Jobs to run one after another
C. Executors to be reused
D. None

✅ **Answer:** A. Multiple jobs to share resources fairly

Q52. YARN client mode vs cluster mode difference?
A. Driver runs on worker node in cluster mode
B. Driver runs on local machine in client mode
C. Both A & B
D. None

✅ **Answer:** C. Both A & B

Q53. Speculative execution in Spark means:
A. Running tasks multiple times
B. Running multiple jobs

C. Running driver twice
D. None

✅ **Answer:** A. Running tasks multiple times

Q54. Which serializer is faster in Spark?
A. JavaSerializer
B. KryoSerializer
C. JSON
D. None

✅ **Answer:** B. KryoSerializer

Q55. Spark dynamic allocation adjusts:
A. Executor count at runtime
B. Cores
C. Memory
D. All

✅ **Answer:** A. Executor count at runtime

Q56. Tungsten's whole-stage code generation improves:
A. JVM bytecode generation
B. Shuffle reduction
C. Network usage
D. None

✅ **Answer:** A. JVM bytecode generation

Q57. Spark UI shows:
A. DAG visualization
B. Stage/Task metrics
C. Shuffle details
D. All

✅ **Answer:** D. All

Q58. Which cluster managers does Spark support?
A. Standalone
B. YARN
C. Mesos
D. Kubernetes
E. All

✅ **Answer:** E. All

Q59. Which parameter controls shuffle file consolidation?
A. spark.shuffle.consolidateFiles
B. spark.shuffle.manager
C. spark.sql.shuffle.partitions
D. None

✅ **Answer:** A. spark.shuffle.consolidateFiles

Q60. The optimal way to persist large DataFrames in Spark is:
A. cache()
B. persist(StorageLevel.MEMORY_AND_DISK)
C. collect()
D. checkpoint()

✅ **Answer:** B. persist(StorageLevel.MEMORY_AND_DISK)
